DEVELOPMENT TASK

BenMaorgal — AI Game-Making Robot MVP

Build a new browser-based child-friendly game creation environment for:

https://www.benmaorgal.com/

The core product idea is:

> **“Talk to a little game-making robot.”**

A child aged approximately 8–15 describes a game in simple natural language. The system selects a suitable small pre-developed game template, configures it using a reusable Game SDK, immediately shows the playable result, and allows the child to continue changing the game through conversational instructions.

The child should not need to know HTML, JavaScript, React, files, repositories, terminals, APIs, or game engines.

The basic UX must feel like:

Tell the robot → Game appears → Play → Ask for a change → Play again

---

0. VERY IMPORTANT: INSPECT THE EXISTING PROJECT FIRST

Before changing code:

1. Inspect the entire existing repository.
2. Identify:
3. framework
4. Next.js version
5. routing system
6. existing pages
7. components
8. APIs
9. public/static assets
10. game-related code

11. database usage, if any
12. environment variables
13. Vercel configuration
14. redirects/rewrites
15. package.json
16. deployment configuration
17. Run the existing project locally.
18. Verify the current Ben’s Brawl Apps application works before making changes.
19. Do NOT delete or rewrite the existing application.
20. Preserve existing functionality.

Create a short internal implementation plan before editing.

---

1. DOMAIN AND ROUTING MIGRATION

The project is deployed through Git to Vercel.

Production domain:

https://www.benmaorgal.com

The new Game-Making Robot application must replace the current application as the primary website.

Required final routing

New application

Main Game-Making Robot landing/workspace:

```
https://www.benmaorgal.com/
```

My Games:

```
https://www.benmaorgal.com/games
```

Individual game editing/play workspace:

```
https://www.benmaorgal.com/games/[gameId]
```

Optional dedicated play route if useful:

```
https://www.benmaorgal.com/play/[gameId]
```

Do not create unnecessary routes if the editor route can handle both editing and playing.

Existing application

The current **Ben's Brawl Apps** application must be preserved.

Move it under:

```
https://www.benmaorgal.com/battle
```

Its existing landing page/current `/game` functionality must become available under `/battle`.

For example, if the existing project currently contains routes like:

```
/game
/game/123
/game/something
```

migrate them logically to:

```
/battle
/battle/123
/battle/something
```

The exact mapping must be based on the existing code discovered during repository inspection.

Important

Do not merely redirect `/battle` to the old page if that creates broken nested paths.

Move/refactor the route structure properly so that all existing internal links and functionality continue working under `/battle`.

---

2. LEGACY URL COMPATIBILITY

After migration, old URLs should not simply break.

Where practical, add redirects.

At minimum:

```
/game → /battle
```

If the existing application has meaningful nested `/game/...` URLs, preserve them through redirects or equivalent mappings where possible.

Example:

```
/game/123456 → /battle/123456
```

Use permanent redirects only when appropriate.

Make sure the new Game-Making Robot does NOT take ownership of `/game`.

The canonical old Brawl application location should become:

```
/battle
```

---

3. PRIMARY PRODUCT OBJECTIVE

Create an MVP demonstrating that a child can create and modify a playable browser game by chatting with a friendly game-making robot.

Example:

Child:

```
Make a game where a cat catches stars.
```

System:

1. understands that this corresponds to the Catch template
2. creates a GameDefinition
3. loads the Catch game using the Game SDK
4. displays it immediately
5. saves it

6. responds:

```
Done! 🐱⭐

Catch the stars!

▶ Play!
```

Child then says:

```
Make the cat faster.
```

System changes only the relevant game property.

Then:

```
Add bombs.
```

System adds dangerous falling objects.

The child can immediately play the modified version.

---

4. FUNDAMENTAL ARCHITECTURAL PRINCIPLE

Do NOT build an unrestricted AI code generator.

Do NOT ask an LLM to rewrite HTML/JS/React code every time the child asks for a change.

Instead build:

```
Child prompt
     ↓
Game Interpreter
     ↓
Game Intent / Game Actions
     ↓
Validation
     ↓
GameDefinition
     ↓
Game SDK
```

```
     ↓
Game Template
     ↓
Playable Game
```

The MVP should use a local deterministic/mock interpreter.

A real AI model will be added later.

The architecture must make replacing the local interpreter with an OpenAI API implementation straightforward.

---

5. GAME LIBRARY

Implement exactly these 10 starter game families.

They must be simple, small, visually understandable, and share as much Game SDK functionality as reasonably possible.

GAME 1 — Catch

Example theme:

```
Cat catches falling stars.
```

Example prompt:

```
Make a game where a cat catches stars.
```

Mechanics:

- player at bottom
- horizontal left/right movement
- objects fall from top
- collision with collectible
- score increases
- target score
- win state

Useful configurable properties:

- player sprite/type
- player speed
- player size
- collectible sprite
- collectible speed
- spawn rate
- points
- background
- target score
- lives

GAME 2 — Dodge

Example:

```
Avoid falling bombs.
```

Mechanics:

- player horizontal movement
- hazards fall
- player avoids hazards
- collision removes life
- survival score or timer
- game over after zero lives

Configurable:

- hazard type
- hazard speed
- spawn rate
- player speed
- player size
- lives
- background
- difficulty

GAME 3 — Jumper

Example:

```
Frog jumps over monsters.
```

Mechanics:

- runner/jumper
- ground
- gravity
- jump
- moving obstacles
- collisions
- score based on distance/obstacles
- game over

Controls:

- keyboard space
- click/tap
- large touch Jump button

GAME 4 — Shooter

Example:

```
Spaceship shoots aliens.
```

Mechanics:

- horizontal player movement
- projectiles
- falling/moving enemies
- enemy/projectile collision
- player/enemy collision
- score
- lives
- win/lose

Controls:

- keyboard left/right
- space/fire
- touch movement buttons
- touch fire button

GAME 5 — Pong

Very small single-player Pong-style game.

Mechanics:

- paddle
- ball
- wall bounce
- paddle collision
- score
- missed ball/lives

Do not attempt network multiplayer.

GAME 6 — Breakout

Mechanics:

- paddle
- ball
- bricks
- bounce
- brick collision
- brick destruction
- score
- win when all bricks disappear

Keep it extremely simple.

GAME 7 — Snake

Mechanics:

- grid
- snake
- food
- directional movement
- growth
- score
- boundary/self collision

Support keyboard and touch arrows.

GAME 8 — Memory

A simple card matching game.

Mechanics:

- cards
- hidden/revealed state
- matching pairs
- number of attempts
- completed pairs
- win condition

Start with simple emoji-based assets.

GAME 9 — Clicker

Example:

```
Click the monster before it disappears.
```

Mechanics:

- object appears
- random position
- click/touch
- object disappears/reappears
- score
- countdown timer

Configurable:

- object
- timeout
- game duration
- target score
- background

GAME 10 — Racer

Simple top-down endless-road style.

Mechanics:

- player car
- left/right movement
- road movement/illusion of scrolling
- obstacle cars
- collisions
- score/distance
- increasing difficulty

Do not build complicated racing physics.

---

6. IMPORTANT: THE 10 GAMES ARE NOT 10 ISOLATED APPS

Implement reusable primitives.

Conceptually:

```
Catch
=
Player
+ HorizontalMovement
+ FallingObjects
+ CollectibleCollision
+ Score

Dodge
=
Player
+ HorizontalMovement
+ FallingObjects
+ HazardCollision
+ Lives

Shooter
=
Player
+ Movement
```

```
+ Projectile
+ Enemy
+ Collision
+ Score

Racer
=
Player
+ HorizontalMovement
+ ScrollingWorld
+ Obstacles
+ Collision

Breakout
=
Paddle
+ Ball
+ Bounce
+ Blocks
+ Collision
```

Reuse systems rather than duplicating logic.

---

7. GAME SDK

Create an internal module such as:

```
src/game-sdk/
```

Suggested structure:

```
game-sdk/
  core/
    GameEngine.js
    GameLoop.js
    GameRuntime.js
    GameDefinition.js

  entities/
    Entity.js
    Player.js
    Enemy.js
    Collectible.js
```

```
    Hazard.js
    Projectile.js
    Ball.js

  systems/
    MovementSystem.js
    CollisionSystem.js
    SpawnSystem.js
    ScoreSystem.js
    LivesSystem.js
    TimerSystem.js
    LevelSystem.js

  physics/
    gravity.js
    collision.js
    bounce.js

  input/
    KeyboardInput.js
    PointerInput.js
    TouchInput.js

  rendering/
    CanvasRenderer.js
    SpriteRenderer.js

  audio/
    SoundManager.js

  utils/
```

Do not over-engineer this hierarchy if a simpler equivalent is cleaner.

The goal is understandable modular code.

---

8. GAME SDK CAPABILITIES

The engine should support the following reusable concepts.

World

Properties such as:

```
{
  width,
  height,
  background,
  gravity,
  boundaries
}
```

Entity

Base concepts:

```
{
  id,
  type,
  x,
  y,
  width,
  height,
  velocityX,
  velocityY,
  sprite,
  visible,
  active
}
```

Player

Support capabilities such as:

- horizontal movement
- vertical movement where needed
- jumping
- firing
- lives/health where required

Object types

Support:

- collectible
- hazard
- obstacle
- enemy
- projectile
- food
- block

---

9. GAME STATE

Standardize common game state.

Example:

```
{
  status: 'ready',
  score: 0,
  lives: 3,
  level: 1,
  elapsedTime: 0,
  isPaused: false,
  isWon: false,
  isLost: false
}
```

Possible statuses:

```
ready
playing
paused
won
lost
```

---

10. GAME DEFINITION

Every generated/configured game must have a serializable structured GameDefinition.

Example:

```
{
  id: 'game_xyz',
  version: 1,

  template: 'catch',

  title: 'Star Cat',

  theme: {
    background: 'space'
  },

  player: {
    type: 'cat',
    size: 60,
    speed: 6
  },

  objects: [
    {
      id: 'star',
      role: 'collectible',
      type: 'star',
      speed: 3,
      spawnRate: 1200,
      points: 1
    }
  ],

  rules: {
    startingLives: 3,
    targetScore: 20
  }
}
```

Create appropriate schema/types/constants.

---

11. VALIDATION

Never pass arbitrary unvalidated data directly into the engine.

Create:

```
validateGameDefinition()
```

Validate:

- known template
- allowed object types
- numeric ranges
- required fields
- valid speeds
- valid spawn rates
- supported sprites
- reasonable lives
- reasonable dimensions
- supported rules

Return safe defaults where reasonable.

Reject impossible definitions gracefully.

This validation layer becomes especially important when a real AI is introduced later.

---

12. GAME ACTION MODEL

Modifying an existing game should normally produce structured actions rather than replacing the whole GameDefinition.

Implement GameAction.

Examples:

```
{
  type: 'SET_PROPERTY',
  path: 'player.speed',
  value: 9
}
```

Example:

```
{
  type: 'ADD_OBJECT',
  object: {
    role: 'hazard',
```

```
    type: 'bomb',
    speed: 4,
    effect: 'loseLife'
  }
}
```

Possible MVP action types:

```
SET_PROPERTY
ADD_OBJECT
REMOVE_OBJECT
CHANGE_THEME
CHANGE_PLAYER
SET_DIFFICULTY
RESET_GAME
```

Add more only if genuinely needed.

---

13. APPLY ACTIONS SAFELY

Create a function/interface conceptually equivalent to:

```
applyGameAction(gameDefinition, action)
```

Process:

```
Current valid GameDefinition
        ↓
GameAction
        ↓
Apply
        ↓
Validate
        ↓
New valid GameDefinition
```

If validation fails:

- preserve previous game
- do not crash
- report friendly error

14. HISTORY

Store human-understandable actions.

Example:

```
⭐ Created Star Cat

🐱 Changed player to cat

💨 Made player faster

💣 Added bombs

❤️ Changed lives to 5
```

Each entry should contain internally:

```
{
  id,
  timestamp,
  prompt,
  action,
  description,
  previousDefinition,
  resultingDefinition
}
```

For MVP, storing snapshots is acceptable because GameDefinitions are small.

15. UNDO

Provide:

```
↩ Undo
```

Undo the latest successful change.

After Undo:

- restore previous GameDefinition
- refresh running game
- keep history logically consistent

The child should never need to understand versions.

Friendly robot example:

```
No problem! ↩️

I changed it back.
```

---

16. LOCAL PROMPT INTERPRETER

For MVP, DO NOT require an external AI account or API key.

Implement a local:

```
interpretPrompt(prompt, currentGame)
```

The interface must later be replaceable by a server AI implementation.

For new games, return something conceptually like:

```
{
  intent: 'CREATE_GAME',
  template: 'catch',
  title: 'Star Cat',
  definitionChanges: {...},
  robotMessage: 'Done! ...'
}
```

For changes:

```
{
  intent: 'MODIFY_GAME',
  actions: [...],
```

```
  robotMessage: 'Your cat is faster now! 🐱💨'
}
```

---

17. NEW-GAME PROMPT RECOGNITION

Recognize flexible keyword patterns.

Examples:

Catch

```
catch
collect
falling stars
catch stars
collect apples
catch bones
```

Dodge

```
avoid
dodge
bomb
avoid monsters
don't get hit
```

Jumper

```
jump
jump over
frog
runner
obstacles
```

Shooter

```
shoot
laser
spaceship
```

```
aliens
fire
```

Pong

```
pong
paddle
ball
```

Breakout

```
break blocks
break bricks
breakout
```

Snake

```
snake
eat apples
snake game
```

Memory

```
memory
match
matching cards
pairs
```

Clicker

```
click
tap
hit monster
catch monster
before it disappears
```

Racer

```
car
race
```

```
racing
drive
avoid cars
```

---

18. SIMPLE SEMANTIC CUSTOMIZATION

Even without AI, support basic theme substitutions.

Example:

```
Make a dog catch bones.
```

should result approximately in:

```
template = catch
player = dog
collectible = bone
```

Example:

```
Make a spaceship catch stars.
```

should reuse Catch with:

```
player = spaceship
collectible = star
background = space
```

Keep the supported vocabulary intentionally limited.

Unknown characters can fall back to generic friendly sprites.

---

19. MODIFICATION PROMPTS

Support at least these modification categories.

Player speed

```
make the player faster
make it faster
make the cat faster
make the car faster
```

Increase within safe limits.

Player slower

```
make it slower
make the player slower
```

Lives

```
give me 5 lives
I want 4 lives
add a life
```

Difficulty

```
make it harder
make the game harder
```

Possible effect:

- object speed +
- spawn rate +
- enemy speed +

Do not make changes absurd.

Easier

```
make it easier
```

Reverse appropriate difficulty parameters.

Add bombs

```
add bombs
add dangerous bombs
```

Where meaningful, add a hazard.

Remove bombs

```
remove bombs
no bombs
```

Player character

Support basic:

```
cat
dog
frog
robot
spaceship
car
hero
```

Background

Support:

```
space
forest
city
ocean
desert
grass
night
```

---

20. CHILD-FRIENDLY FALLBACK

If prompt is not understood:

Do NOT create a random game.

Robot says something like:

```
Hmm… 🤖

I’m not sure how to do that yet.

Try:

⭐ “Make me faster”
💣 “Add bombs”
❤️ “Give me 5 lives”
🌌 “Make the background space”
```

---

21. MAIN LANDING PAGE

The new root:

```
/
```

must be the Game-Making Robot application.

Design a visually exciting child-friendly landing page.

Suggested hero:

```
🤖🎮

BUILD YOUR OWN GAME!

Tell me what you want to play.
I'll help you make it!

[ ✨ CREATE A GAME ]

[ 🎮 MY GAMES ]

Already have games?
Continue building!
```

Also provide a small link:

```
⚔️ Ben's Brawl Apps
```

leading to:

```
/battle
```

Do not make the old application the visual focus.

---

22. CREATE GAME FLOW

Click:

```
✨ CREATE A GAME
```

Open creation workspace.

Could use:

```
/games/new
```

or create an ID immediately and navigate to:

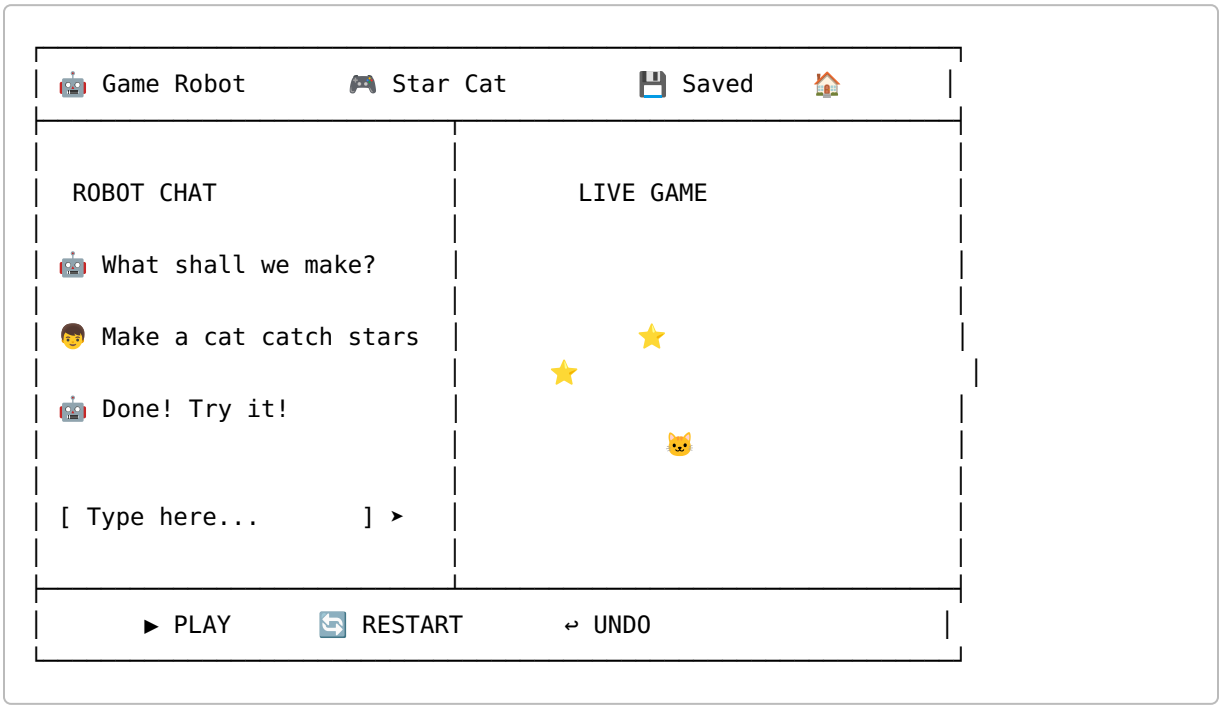
```
/games/[id]
```

Choose the cleaner architecture.

---

23. WORKSPACE UI

Desktop layout:



Approximately 30–40% chat and 60–70% game area.

---

24. MOBILE / TABLET

On narrow screens:

```
Game
↓
Controls
↓
Robot Chat
```

or Chat → Game depending on usability.

The game must remain comfortably playable.

Touch buttons should be large.

Minimum reasonable touch target around 44px.

---

25. CHAT DESIGN

Robot messages:

- friendly
- short
- encouraging
- nontechnical

Example:

```
🤖 Great idea!

I made a cat that catches stars. 🐱⭐

Get 20 stars to win!
```

Do NOT say:

```
I generated a GameDefinition using the CatchTemplate component.
```

---

26. CHAT MESSAGE HISTORY

For each game, persist conversation-like records locally.

Suggested:

```
{
  id,
  role: 'user' | 'robot',
  text,
```

```
  timestamp
}
```

Display a scrollable chat.

Auto-scroll to latest message.

---

27. NO FAKE AI DELAYS

Do not deliberately make the child wait several seconds.

The local interpreter should respond immediately.

A very short visual animation such as:

```
🤖✨ Building...
```

for a few hundred milliseconds is acceptable if it improves UX.

Do not unnecessarily delay functionality.

---

28. LIVE GAME

The right-side game should render in a dedicated GameViewport component.

Suggested:

```
<GameViewport definition={gameDefinition} />
```

Changing GameDefinition should reset/update the runtime safely.

Provide:

```
▶ PLAY
🔄 RESTART
```

If the game is over:

```
🎉 YOU WIN!
```

or:

```
💥 GAME OVER
```

with:

```
PLAY AGAIN
```

---

29. VISUAL ASSET STRATEGY

For MVP avoid requiring external copyrighted game assets.

Do NOT use Brawl Stars characters or other copyrighted game characters in the new Game Robot.

Use:

- emoji
- CSS illustrations
- very simple original SVG-style shapes
- lightweight original generic character graphics

Characters may include:

```
🐱 cat
🐶 dog
🐸 frog
🤖 robot
🚀 spaceship
🏎️ car
👾 alien
⭐ star
💣 bomb
🍎 apple
```

An asset library can be expanded later.

---

30. MY GAMES PAGE

Route:

```
/games
```

Show:

```
MY GAMES

[ + CREATE NEW GAME ]

┌──────────────┐
│ ⭐🐱         │
│ STAR CAT     │
│              │
│ ▶ PLAY       │
│ ✏ EDIT       │
│ 🗑️ DELETE     │
└──────────────┘
```

Game cards should contain:

- simple visual preview
- game title
- template name/icon
- last modified
- Play
- Edit
- Delete

---

31. DELETE CONFIRMATION

Do not immediately delete when a child hits Delete.

Use:

```
Delete “Star Cat”?

Your game will disappear.
```

```
[ CANCEL ] [ 🗑️ DELETE ]
```

---

32. GAME RENAMING

Allow title rename from workspace or My Games.

Examples:

```
Star Cat
Space Attack
Ben's Super Racer
```

Validate reasonable maximum length.

---

33. LOCAL STORAGE

For this MVP use browser localStorage.

Create a clean persistence abstraction.

Do not directly spread localStorage calls through many components.

Example:

```
gameRepository/
    getGames()
    getGame(id)
    saveGame(game)
    deleteGame(id)
```

Later this repository will be replaceable with server/database implementation.

Use a namespace/version key such as:

```
benmaorgal-games-v1
```

---

34. GAME DATA MODEL

Suggested structure:

```
{
  id,
  title,
  template,

  definition,

  messages: [],

  history: [],

  createdAt,
  updatedAt
}
```

---

35. SAVE BEHAVIOR

Auto-save changes.

Show a subtle state:

```
💾 Saved
```

Do not require children to understand manual saving.

---

36. DEFAULT EXAMPLE GAME

For first launch, consider displaying an unsaved demonstration such as:

```
🐱 Star Catcher
```

but do not automatically pollute My Games unless the user explicitly creates/saves it.

Alternatively go directly to friendly robot prompt.

Choose whichever produces the cleaner UX.

---

37. 10 QUICK START EXAMPLES

In Create Game, provide visual starter cards in addition to free text.

Example:

```
⭐ Catch
💣 Dodge
🐸 Jump
🚀 Shoot
🏓 Pong

🧱 Breakout
🐍 Snake
🧠 Memory
🎯 Click
🏎️ Race
```

Clicking one should create a basic version of that game.

This allows children who do not know what to type to start instantly.

---

38. ROBOT SUGGESTIONS AFTER CREATION

After a game is created, show 2–4 relevant suggestions.

Example Catch:

```
Try saying:

💨 “Make me faster”
💣 “Add bombs”
❤️ “Give me 5 lives”
🌌 “Make it space”
```

Example Shooter:

```
Try saying:

👾 “Add more aliens”
🚀 “Make my spaceship faster”
❤️ “Give me 5 lives”
⚡ “Make it harder”
```

These suggestions are educational scaffolding.

---

39. OPTIONAL QUICK-ACTION CHIPS

Under robot messages, it is acceptable to provide clickable chips:

```
[ 💨 Faster ]
[ 💣 Add bombs ]
[ ❤️ 5 lives ]
```

Clicking a chip should internally behave like sending the associated prompt.

Do not replace natural-language chat with buttons.

---

40. GAME TEMPLATE METADATA

Each template should expose metadata.

Example:

```
{
  id: 'catch',
  name: 'Catch',
  icon: '⭐',
  description: 'Catch falling objects',
  examplePrompt: 'Make a cat catch stars',
  supportedActions: [...]
}
```

Use this to populate template selection UI and prompt suggestions.

---

41. DIFFICULTY

Where relevant support:

```
easy
normal
hard
```

Do not expose complex numeric physics to children.

Internally map difficulty to safe parameters.

Example:

```
easy: {
  speedMultiplier: 0.8,
  spawnMultiplier: 0.8
}

normal: {...}

hard: {
  speedMultiplier: 1.3,
  spawnMultiplier: 1.2
}
```

---

42. GAME ENGINE LOOP

For Canvas games use:

```
requestAnimationFrame
```

Separate:

```
update()
render()
```

Handle delta time reasonably.

Do not tie game physics directly to monitor refresh rate.

Pause loops when appropriate.

Clean up requestAnimationFrame and listeners when components unmount.

---

43. INPUT ARCHITECTURE

Normalize keyboard and touch input.

Games should not each independently reimplement keyboard listeners if avoidable.

Provide common abstractions.

Example concepts:

```
left
right
up
down
jump
fire
```

Map keyboard/touch to these logical actions.

---

44. COLLISION

Simple AABB collision is sufficient for most MVP games.

Do not build a complex physics engine.

For circular ball games, simple appropriate ball collision logic is fine.

---

45. SOUND

Prepare SoundManager/hooks but keep sound optional.

If using sounds:

- subtle
- short
- easily muted

Provide:

```
🔊 / 🔇
```

Do not depend on sound for understanding game state.

---

46. ACCESSIBILITY

At minimum:

- adequate contrast
- keyboard usable controls
- visible focus where practical
- buttons with text/aria labels
- do not convey important game UI by color alone

---

47. CHILD SAFETY / TECHNICAL SAFETY

Even though MVP uses no real AI:

Design future AI boundary safely.

Future model output should be:

```
structured GameIntent / GameActions
```

not arbitrary JavaScript.

Never architect the main application around:

```
eval(aiOutput)
```

or:

```
new Function(aiOutput)
```

Do not execute arbitrary AI-generated code.

---

48. FUTURE AI ADAPTER

Create interface structure such as:

```
class GameInterpreter {
  async interpret(prompt, context) {}
}
```

or simple functions/modules.

Implement:

```
LocalGameInterpreter
```

now.

Make future addition easy:

```
OpenAIGameInterpreter
```

Do NOT implement OpenAI yet unless there is already relevant infrastructure in the repository and doing so is completely isolated.

The MVP must run without API keys.

---

49. FUTURE USER SYSTEM — ARCHITECTURE ONLY

Do not implement authentication now.

But do not design local game IDs in a way that prevents later ownership.

Future model:

```
User
 ├─ id
 ├─ nickname
 └─ passwordHash

Game
 ├─ id
 ├─ ownerId
 └─ ...

Friendship
Invitation
GamePermission
```

Eventually:

OWNER:

```
create
play
edit
delete
share
```

FRIEND:

```
play/read-only
```

No implementation required now.

---

50. FUTURE VOICE

The chat input design must reserve a microphone control.

Example:

```
[ 🎤 ] [ Tell me what to change...        ] [ ➤ ]
```

Do not implement voice recognition in this MVP unless extremely trivial and isolated.

It will later support Hebrew and English.

---

51. FUTURE EDUCATIONAL MODE

Prepare conceptual space for later buttons:

```
🧠 HOW DOES MY GAME WORK?
```

and:

```
👩‍💻 SHOW THE CODE
```

Do not implement full code education yet.

A future educational representation may show:

```
PLAYER
🐱 moves left/right

OBJECT
⭐ falls

RULE
🐱 + ⭐ = +1 point
```

---

52. EXISTING BRAWL APP MIGRATION

This is a critical part of the task.

After understanding current routing:

1. Move the existing Brawl application from its current primary location to:

```
/battle
```

1. Preserve all components and logic.

2. Update internal navigation.

3. Update relative route references.

4. Update links pointing to old `/game` paths.

5. Check API route assumptions.

6. Check asset paths.

7. Check browser refresh/direct navigation to nested routes.

8. Verify Vercel routing.

9. Add appropriate legacy redirects.

---

53. BATTLE ENTRY FROM NEW SITE

On the new main landing page add a secondary area such as:

```
Already using Ben's Brawl Apps?

⚔️ OPEN BATTLE APPS
```

Link:

```
/battle
```

This keeps the existing product easily accessible without confusing it with the new Game Robot.

---

54. NAVIGATION

Suggested new main navigation:

```
🤖 CREATE
🎮 MY GAMES
⚔️ BATTLE APPS
```

Keep it small.

Do not create a complicated navigation menu.

55. CHILDISH DESIGN — BUT NOT BABYISH

Target range:

```
8–15 years
```

The UI should feel:

- fun
- modern
- game-like
- colorful
- friendly

Avoid:

- toddler graphics
- excessive baby language
- very long explanatory texts

Think:

```
simple game dashboard + friendly robot
```

rather than kindergarten software.

---

56. ROBOT CHARACTER

Create a simple original robot mascot using CSS/SVG/emoji-inspired original graphics.

Robot may have states:

```
idle
thinking
happy
oops
celebrate
```

Keep implementation lightweight.

Examples:

Game created:

Win:

Error:

---

57. ANIMATIONS

Use subtle animations for:

- robot response
- game card hover
- buttons
- stars/confetti after creation/win
- history update

Avoid heavy libraries unless existing project already uses one.

Prefer CSS animations.

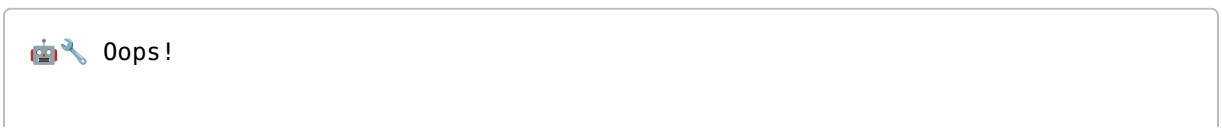
---

58. ERROR BOUNDARY

Game runtime errors must not take down the entire React application.

Create appropriate error isolation.

If a game fails:



```
Something went wrong with this game.

[ RESTART GAME ]
```

Log technical details in console in development.

---

59. EMPTY STATES

My Games empty state:

```
You haven't made a game yet! 🎮

Tell the robot your first idea.

[ ✨ MAKE MY FIRST GAME ]
```

---

60. STORAGE CORRUPTION

If localStorage contains invalid game data:

- do not crash
- skip/recover invalid entries where possible
- preserve valid entries
- log diagnostic information

---

61. PROJECT STRUCTURE

After inspecting existing repository, adapt rather than blindly replacing it.

A possible structure:

```
src/
  app/
    page.js

    games/
      page.js
      new/
```

```
      page.js
    [gameId]/
      page.js

  battle/
    ...

components/
  robot/
    RobotAvatar.js
    RobotChat.js
    ChatMessage.js
    PromptInput.js
    SuggestionChips.js

  games/
    GameViewport.js
    GameControls.js
    GameCard.js
    GameHistory.js

  layout/
  ui/

game-sdk/
  core/
  entities/
  systems/
  physics/
  input/
  rendering/
  audio/

game-templates/
  catch/
  dodge/
  jumper/
  shooter/
  pong/
  breakout/
  snake/
  memory/
  clicker/
  racer/

game-interpreter/
  LocalGameInterpreter.js
  promptPatterns.js
```

```
    actionFactory.js

  game-data/
    schema.js
    validation.js
    actions.js

  repositories/
    localGameRepository.js

  utils/
```

Use existing project conventions where superior.

---

62. DO NOT CREATE DUPLICATE ENGINES

If existing Brawl game contains useful generic utilities, inspect whether they can safely be reused.

However:

Do not tightly couple the new Game SDK to old Brawl-specific code.

The new Game SDK should be logically independent.

---

63. CODE STYLE

Use:

- modern ES6+
- `import` / `export`
- clear naming
- small modules
- understandable code
- comments only where useful

Do NOT use CommonJS `require` unless required by existing tooling.

Do NOT add semicolons to JavaScript unless required.

Avoid unnecessary TypeScript conversion if the existing project is JavaScript.

Do not migrate the entire project to another framework.

64. DEPENDENCIES

Keep dependencies minimal.

Before adding a package ask internally:

```
Can this reasonably be implemented with existing code/platform APIs?
```

Do not install a game engine such as Phaser for this MVP unless analysis of the existing repository reveals an extremely compelling reason.

Prefer the custom small SDK because understanding and extensibility are part of the project objective.

---

65. PERFORMANCE

Target:

- fast initial page
- immediate template creation
- smooth 60fps animation where practical
- no large image assets
- no enormous JavaScript libraries
- no unnecessary network calls

---

66. VERCEL COMPATIBILITY

Ensure:

```
npm run build
```

works successfully before completion.

Check:

- dynamic routes
- redirects
- client/server component boundaries
- browser-only APIs such as localStorage

- Canvas code
- window/document references
- hydration issues

Do not access localStorage during server rendering.

---

67. DOMAIN ASSUMPTION

Do not hard-code:

```
localhost
```

or unnecessary absolute production URLs.

Use relative application routing where possible.

Production is:

```
https://www.benmaorgal.com
```

Vercel/domain configuration is already expected to point there.

Do not alter DNS.

---

68. TESTING REQUIREMENTS

Manually verify all ten game templates.

For every template test:

```
Create
Play
Restart
Edit
Persist
Reload page
Open again
Delete
```

Also test Undo for representative modification types.

---

69. REQUIRED PROMPT TESTS

At minimum test these exact prompts.

Creation

```
Make a game where a cat catches stars
```

Expected:

```
Catch
cat
stars
```

```
Make a game where I avoid bombs
```

Expected:

```
Dodge
```

```
Make a frog jump over monsters
```

Expected:

```
Jumper
```

```
Make a spaceship shoot aliens
```

Expected:

```
Shooter
```

```
Make Pong
```

Expected:

```
Pong
```

```
Make a game where I break bricks
```

Expected:

```
Breakout
```

```
Make a snake that eats apples
```

Expected:

```
Snake
```

```
Make a matching cards game
```

Expected:

```
Memory
```

```
Make a game where I click the monster
```

Expected:

```
Clicker
```

```
Make a car racing game
```

Expected:

```
Racer
```

---

70. REQUIRED MODIFICATION TEST

Use Catch.

Start with:

```
Make a game where a cat catches stars
```

Then sequentially:

```
Make the cat faster
```

Verify speed increases.

Then:

```
Add bombs
```

Verify bombs appear and cost lives.

Then:

```
Give me 5 lives
```

Verify 5 lives.

Then:

```
Make it harder
```

Verify reasonable difficulty increase.

Then:

```
Make the background space
```

Verify visual change.

Then Undo.

Verify the latest change disappears.

---

71. PERSISTENCE TEST

Create:

```
Star Cat
```

Modify it.

Refresh browser.

Verify:

- game remains
- latest GameDefinition remains
- title remains
- history remains
- messages remain

---

72. MULTIPLE GAME TEST

Create at least three games:

```
Star Cat
Alien Attack
Super Racer
```

Verify all appear independently in My Games.

Editing one must not modify another.

---

73. RESPONSIVE TEST

Test approximately:

```
Desktop 1920x1080
Laptop 1366x768
Tablet
Mobile
```

Focus on usability rather than pixel perfection.

---

74. EXISTING BATTLE APP REGRESSION TEST

After migration verify:

```
/battle
```

loads correctly.

Test major existing Brawl functionality.

Confirm old application data/features have not been unintentionally removed.

Test direct browser reload while on `/battle`.

Test important nested pages.

---

75. LEGACY ROUTE TEST

Verify:

```
/game
```

moves/redirects correctly to:

```
/battle
```

Test any important old nested URLs discovered during inspection.

---

76. BUILD TEST

Before considering task complete run:

```
npm run build
```

Resolve all production build errors.

Also check meaningful lint issues if linting exists.

---

77. DO NOT DO IN THIS PHASE

Do NOT implement:

- production user registration
- password authentication
- database
- friends
- invitations
- public game sharing
- multiplayer
- payment
- subscriptions
- OpenAI API dependency
- image generation API
- generated arbitrary JavaScript

- Google login
- GitHub login
- voice backend
- complex level editor
- marketplace
- social network
- Brawl Stars cloning
- copyrighted characters

These belong to later stages.

---

78. PREPARE FOR STAGE 2

Architecture should make Stage 2 relatively straightforward.

Stage 2 is expected to add:

```
User registration
Unique nickname
Password
Cloud database
Cloud games
Real AI interpreter
Voice prompts
Hebrew/English support
Friends
Invitations
Read-only access
```

Therefore maintain clean abstraction boundaries now.

---

79. IMPORTANT FUTURE AI CONTEXT

When real AI is added, it should receive context approximately like:

```
{
  prompt,
  currentGameDefinition,
  currentTemplate,
  supportedTemplates,
  supportedActions,
```

```
  allowedAssets
}
```

And return structured output approximately:

```
{
  intent: 'modify_game',

  actions: [
    {
      type: 'SET_PROPERTY',
      path: 'player.speed',
      value: 8
    }
  ],

  response: 'Your cat is faster now! 🐱💨'
}
```

This must pass validation before execution.

Design current interfaces with this future flow in mind.

---

80. MVP SUCCESS CRITERIA

The MVP is successful if a child can open:

```
https://www.benmaorgal.com
```

and without explanation:

1. understand that he can make a game
2. type:

```
Make a game where a cat catches stars
```

1. see a playable game
2. play it
3. type:

```
Add bombs
```

1. immediately play the modified version
2. type:

```
Make it faster
```

1. see the change
2. press Undo
3. see the previous version
4. return to My Games
5. reopen the game

At the same time, the old existing application must remain available at:

```
https://www.benmaorgal.com/battle
```

---

81. IMPLEMENTATION ORDER

Follow approximately this order.

Phase A — repository understanding

- inspect
- run
- understand current app
- document route dependencies

Phase B — migrate existing app

- move Brawl Apps to `/battle`
- fix links/routes
- add legacy redirect
- regression test

Do this before replacing `/`.

Phase C — Game SDK foundation

Implement:

- runtime
- loop
- input
- entities
- collision
- state

Phase D — first vertical slice

Implement only:

```
Catch
```

Then make this complete flow work:

```
prompt
→ definition
→ game
→ play
→ modification
→ history
→ undo
→ save
```

Do not build all ten games before validating this architecture.

Phase E — remaining templates

Add:

```
Dodge
Jumper
Shooter
Pong
Breakout
Snake
Memory
```

```
Clicker
Racer
```

using shared SDK functionality.

Phase F — application UX

Complete:

- landing
- robot chat
- My Games
- game cards
- history
- responsive design

Phase G — hardening

- errors
- invalid storage
- route testing
- responsiveness
- build
- Battle regression

---

82. IMPORTANT DEVELOPMENT RULE

After Phase D, stop internally and evaluate:

```
Is the Catch implementation reusable enough that the next games can share the
engine?
```

If not, refactor BEFORE implementing the other nine templates.

Do not create ten copies of similar game loops.

---

83. FINAL DOCUMENTATION

When implementation is complete, update/create README documentation containing:

Architecture

Explain:

```
Game Robot
Game Interpreter
GameDefinition
GameAction
Game SDK
Templates
Persistence
```

Routes

Document:

```
/                    Game Robot
/games               My Games
/games/[id]          Game workspace
/battle              existing Ben's Brawl Apps
```

and any additional routes actually implemented.

Adding a template

Explain exactly how a developer can add Game #11.

Adding a new action

Explain how to add a new command type.

Future AI integration

Explain where the real AI adapter should be connected.

---

84. FINAL REPORT

At completion provide a concise development report containing:

1. What was changed.
2. Existing app migration details.

3. New routes.
4. Game SDK structure.
5. All 10 implemented templates.
6. Supported prompt commands.
7. Persistence approach.
8. Known limitations.
9. Files/modules most important for future development.
10. Exact commands to run locally.
11. Confirmation that `npm run build` passes.
12. Any Vercel considerations.
13. Recommended next development stage.

---

85. CRITICAL PRINCIPLES

Throughout development preserve these principles:

Principle 1

The child talks to a robot, not to a programming IDE.

Principle 2

Games come from reusable templates and SDK capabilities.

Principle 3

Natural-language changes modify structured configuration.

Principle 4

Do not generate unrestricted executable code.

Principle 5

Every successful modification is reversible.

Principle 6

The game should become playable immediately.

Principle 7

Use the simplest implementation that produces a fun result.

Principle 8

The existing Ben's Brawl Apps must continue working.

Principle 9

The new architecture must be ready for real AI, accounts and cloud persistence later.

Principle 10

The first priority is the experience:

"I told the robot what game I wanted — and it made my game!"